PLAYBACK

Waypoint on Power Platform

A capacity planner, generated from source files in 48 hours.

Craig Mitchinson · Intelligent Process Automation · 21 September 2026

01

What we built

WHAT WE BUILT

The result

A working capacity planner on Dataverse, generated end to end.

Generated and imported by script. Nothing on this slide was laid out by hand.

48 hours *from a model file to an app that writes real records*

10
screens
all generated from source

27
tables
319 seed rows, loaded in 57 seconds

925
named formulas
bound offline before every import

21
preflight checks
standing between a change and an import

26
logged builds
each with its claim and its actual

The ambition

Build a Power Platform app from source files rather than by clicking it together.

The three artefacts the build starts from.

- Workbook: the original, and still the numbers we answer to
- React prototype: the design and the engine, written down
- Model file: 27 tables, 209 columns, 43 relationships

02

How it works

HOW IT WORKS

The setup

Work moves through named roles, and each role runs on the cheapest model that can do it.

Roles by the model they run on, from the runbook's own table.

Orchestrator — *picks the role and the model from the table, not by feel; edits nothing itself*

CHEAPEST MODEL

reports, never judges

- **scribe** — keeps the written record
- **runner** — runs commands, reports verbatim
- **explore** — finds where something lives
- **test** — runs the suite, returns failures

THE DEFAULT

builds and reviews

- **worker** — one screen, or one concern
- **reviewer** — reads against the brief

ESCALATED

only where the table says

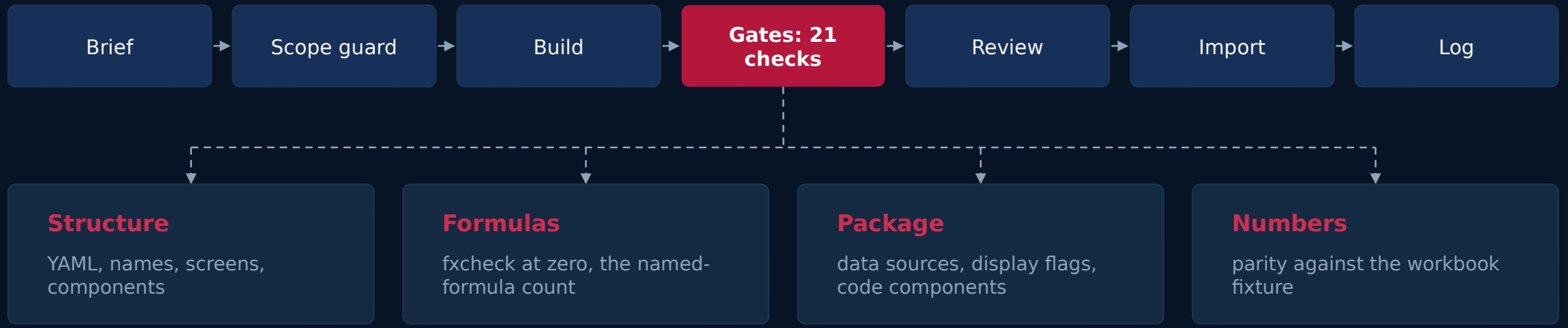
- **worker** — typed Power Fx, the C# tools
- **reviewer** — write paths, security, a retry

Studio: the last word

How a unit moves

Every change follows the same path, and the gates are the same every time.

The pipeline, with the 21 preflight checks grouped by what they protect.



A failed check stops the import. A new class of defect becomes a new check the same day.

Rules, not patches

A defect that can return has not been fixed.

Defect, then the rule and the check it became.

- Connections lost on push: refresh the skeleton first, and refuse to pack without data sources
- Unknown property PA2108: a per-control deny list, read before packing
- Code component dropped: compare the whole package, not the part we author
- Numbers drifting from the workbook: parity runs as a gate

03

What it proves

Proof by computation

The only proof a number is right is running the formula that produces it.

What parity covers, and what it does not.

> Net 221.02 member weeks matched the workbook, and so did every squad figure. Parity also found a real defect nobody had seen: team-held capacity attributed to no one.

- 17 fixture cases, tolerance 0.005, pinned to the workbook's own day
- Covers the Command band, the squad figures, delivery and benefit
- Does not cover live rows, delegation, security roles or the clock
- A gate before import, not a substitute for opening the app

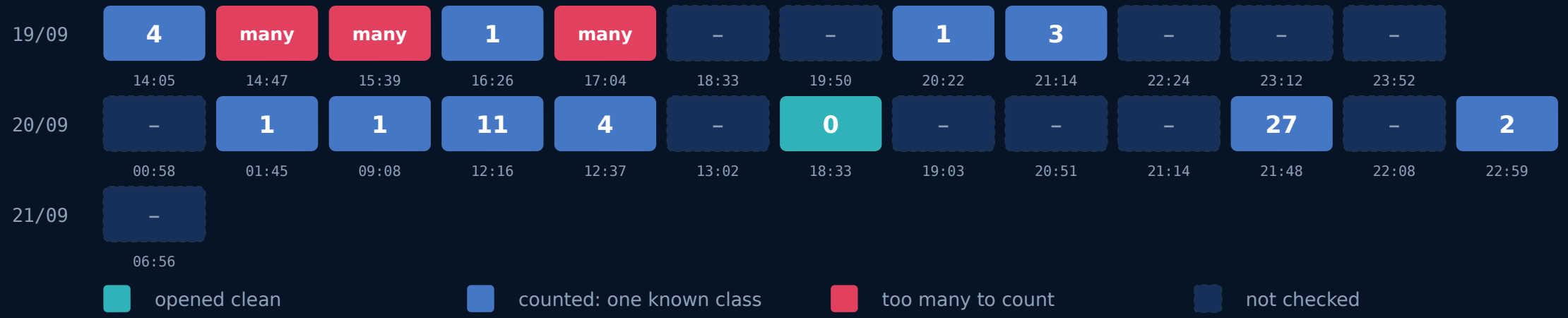
WHAT IT PROVES

The evaluation log

A big number now means one known class, not many unknown problems.

Every import, coloured by what Studio reported.

> The first afternoon's errors were too many to count and had nothing in common. Every count since is a single class: the 27 is one property on 27 controls, fixed once, and a check now fails before an import can repeat it.



Source: power/tests/evaluation-log.md, read at build time. 26 imports. Nobody looking is recorded as not checked, never as clean.

What holds, and what is open

The generated path is proven offline and partly proven in the browser.

Run more than once, against open items with an owner.

HOLDS

- The solution generated from one model file, imported as a single unit
- Seed data from the app's own export, repeatable and byte-identical
- The YAML round trip: download, write, pack, import, open, with no rewritten formula
- The offline checker reproducing Studio's own messages in seconds

OPEN

- A closed-source platform: some property names are only knowable by trying them
- Checker gaps: a gate cannot see what it cannot compile
- Live saves: two of the forms have written to Dataverse, the rest are untried
- Security roles: portal work, and the pipeline cannot do it
- The requester app: designed, parked behind a flag, not yet decided

04

What it means

What the next app costs

The next app inherits the pipeline, not just the lessons.

The estimate, with its assumptions on the slide.

- Honest estimate for a comparable app: days rather than weeks
- The first day goes on the model file; that part does not compress
- The expensive part was Studio round trips, and the checker removes most of them
- Assumes the same environment, the same control set, and a model file already written
- Assumes the reference corpus and the checkers carried over unchanged

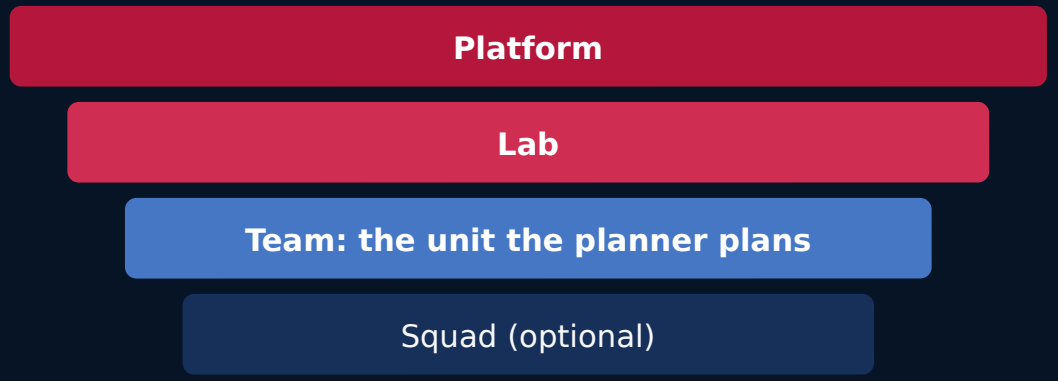
WHAT IT MEANS

The platform model

One journey, from the request arriving to the benefit landing.

The hierarchy, and the journey from front door to benefit.

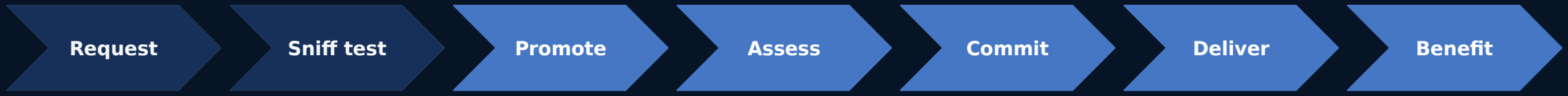
HIERARCHY



TODAY

A request arrives on a form, somebody decides, and the thread ends in a mailbox. The planner never sees the request that became the work.

FRONT DOOR TO BENEFIT



Shaded: already built. The first two steps are the front door, designed and parked behind a flag. A promoted request keeps its link to the work item, so the register answers where the work came from.

WHAT IT MEANS

The decision, and next steps

The near work is finishing what is proven offline but not yet proven in the browser.

What happens next, and what is needed from the room.

THE DECISION

Does the front door become the next unit of work? That is what turns a planner into a system that starts at the request rather than at the work item.

01 Add the four hierarchy tables in Studio and turn the front door on

02 Exercise the remaining live saves in the player

03 Two security roles, mapped to Entra groups

04 Decide on the requester app as a separate surface

05 Take the pipeline to the next app on the platform

05

Appendix: the method

Context discipline

Knowledge lives in files, not in anyone's memory of a conversation.

Where each kind of truth is kept.

- power/RUNBOOK.md: the contract every unit gets verbatim
- power/canvas/columns.json: columns, read from the imported solution
- CONTROLS.md: control names, captured out of Studio
- The skeleton: downloaded fresh before every build

Studio is the last word

Property names are settled by the platform, never by a brief.

What PA2108 taught, and where that knowledge sits now.

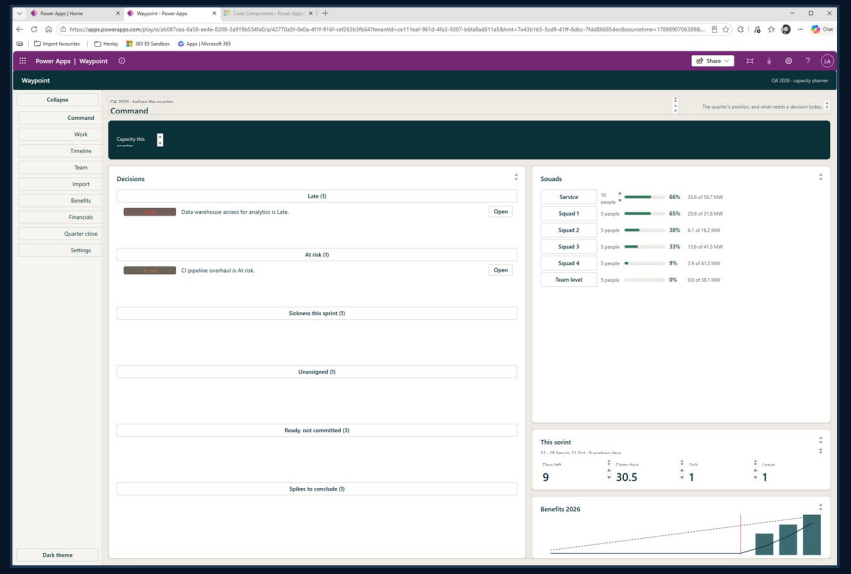
- The sample screen: one of every control, placed in Studio and read back
- CONTROLS.md: identifiers and versions as Studio emits them
- control-properties.json: what each control type refuses
- Display labels in the docs are not the property names in the file

The scroll-bar hunt

Diagnose from the pixels, not from the assumption.

The same screen, before the swap and after it.

BEFORE



AFTER

Same screen, from the current build: to add

Command, 20 September 09:12. Scroll arrows on text that fits, and the capacity band clipped mid-word. All 641 text controls were swapped in one unit; the modern Text control is now on the deny list, so the class cannot return.

Workers that say no

A refusal is the system working, not the system failing.

What the guard refuses, and what it asks for instead.

- More than one screen file in scope: split by screen
- More than one concern bundled: split by concern
- No turn budget given: the brief comes back unstarted
- A partial result reported beats a silent overrun

Drift and hallucination

Meaning drifts at the hand-offs, and each place needs its own guard.

Each drift, and what stops it.

- Display labels read as property names: capture the token from Studio
- A claim standing in for an actual: gates compile, and the log records both
- A brief scoped by implication: explicit file scope and a do-not-touch list
- A later unit undoing an earlier decision: conventions live in the runbook

Decisions made with the user, not for them

Decide the product in words before building the screen twice.

Four of the ten, with the reasoning kept.

- The spine: stage on Work, assess on the item, decide on Command, close on Quarter close
- No booking concept: people are assigned from the work item
- Non-work time is one typed absence form, for one person or many
- Timeline is read-only, so assignment stays in one place